МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Нижегородский государственный технический университет
им. Р.Е. Алексеева»

Арзамасский политехнический институт (филиал)

**ОТЧЕТ**

о выполнении практической работы № 3
по дисциплине «Основы тестирования программного обеспечения»

Тема: «Модульное тестирование класса AuthManager с использованием pytest»

Выполнил: студент гр. АЗИС 23-1
Рубцов М.И.

Проверил: преподаватель
Комаров А.О.

Арзамас, 2026

## 1 Цель работы

Целью работы является освоение фреймворка pytest для модульного тестирования: написание базовых, параметризованных тестов и тестов исключений, использование фикстур для подготовки тестовой базы данных, применение меток для классификации и отбора тестов, пропуск тестов, запуск с ограничением числа неудач, а также проверка уязвимости кода к SQL-инъекциям.

## 2 Задание

Требуется протестировать класс `AuthManager`, который управляет пользователями (регистрация, аутентификация, удаление, получение по идентификатору), подсчитывает пользователей по странам и переводит средства между ними. В тестах необходимо продемонстрировать: три базовых теста, три параметризованных, два теста исключений, фикстуру базы данных, не менее двух меток, один пропускаемый тест и обязательный тест на SQL-инъекции; запустить тесты с параметром `--maxfail`; получить снимки экрана успешного и неуспешного прогонов. Текст тестируемого класса взят из методических указаний без изменения логики (листинг 1); в нем намеренно оставлена подстановка параметров в SQL через `str.format` – учебная уязвимость, наличие которой и должны выявить тесты.

## 3 Теоретические сведения

pytest – фреймворк тестирования для Python, обнаруживающий тесты по именам `test_*.py` и `test_*` и использующий обычный оператор `assert` с подробной диагностикой при падении. Фикстуры (`@pytest.fixture`) описывают подготовку и освобождение ресурсов; конструкция `yield` разделяет setup и teardown, а зависимости между фикстурами задаются параметрами функций. Параметризация (`@pytest.mark.parametrize`) запускает один тест для набора входных данных, каждая комбинация становится отдельным тестовым узлом с собственным идентификатором. Проверка исключений выполняется контекстным менеджером `pytest.raises` с необязательным параметром `match` для проверки текста сообщения. Метки

(`@pytest.mark.<имя>`) классифицируют тесты и позволяют отбирать их опцией `-m`; пользовательские метки регистрируются в `pytest.ini`. Пропуск тестов задается декораторами `skip` и `skipif`. Опция `--maxfail=N` останавливает сессию после N неудач, `--tb=short` сокращает трассировку.

SQL-инъекция – внедрение фрагментов SQL через пользовательские данные, подставляемые в текст запроса без экранирования. Защита – параметризованные запросы (`execute("... WHERE username = ?", (username,))`), при которых драйвер передает значения отдельно от текста команды.

## 4 Тестируемый класс

Класс размещен в модуле `auth_manager.py` (листинг 1). База данных – SQLite; таблица `users` создается в конструкторе. Все методы формируют SQL конкатенацией строк, что делает их уязвимыми к инъекциям.

Листинг 1 – auth_manager.py – тестируемый класс

```
import sqlite3


class AuthManager:
    def __init__(self, connection):
        self.connection = connection
        self.create_tables()

    def create_tables(self):
        with self.connection:
            self.connection.execute("""
                CREATE TABLE IF NOT EXISTS users (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    username TEXT NOT NULL UNIQUE,
                    password TEXT NOT NULL,
                    country TEXT NOT NULL,
                    balance REAL NOT NULL
                )
            """)

    def register_user(self, username, password, country, balance):
        with self.connection:
            self.connection.execute("""
                INSERT INTO users (username, password, country, balance)
                VALUES ('{}', '{}', '{}', {})
            """.format(username, password, country, balance))  # уязвимость: SQL-
инъекция

    def authenticate_user(self, username, password):
        cursor = self.connection.cursor()
        cursor.execute("""
            SELECT * FROM users
            WHERE username = '{}' AND password = '{}'
        """.format(username, password))  # уязвимость: SQL-инъекция
```

```
        return cursor.fetchone()

    def delete_user(self, user_id):
        with self.connection:
            self.connection.execute("""
                DELETE FROM users WHERE id = {}
            """.format(user_id))  # уязвимость: SQL-инъекция


    def get_user_by_id(self, user_id):
        cursor = self.connection.cursor()
        cursor.execute("""
            SELECT * FROM users WHERE id = {}
        """.format(user_id))  # уязвимость: SQL-инъекция
        return cursor.fetchone()


    def count_users_by_country(self, country):
        cursor = self.connection.cursor()
        cursor.execute("""
            SELECT COUNT(*) FROM users WHERE country = '{}'
        """.format(country))  # уязвимость: SQL-инъекция
        return cursor.fetchone()[0]


    def transfer_balance(self, from_user_id, to_user_id, amount):
        with self.connection:
            # Проверяем, достаточно ли средств
            cursor = self.connection.cursor()
            cursor.execute("SELECT balance FROM users WHERE id =
{}".format(from_user_id))  # уязвимость: SQL-инъекция
            from_balance = cursor.fetchone()[0]


            if from_balance < amount:
                raise ValueError("Insufficient funds")


            # Выполняем перевод
            self.connection.execute("""
                UPDATE users SET balance = balance - {} WHERE id = {}
            """.format(amount, from_user_id))  # уязвимость: SQL-инъекция
            self.connection.execute("""
                UPDATE users SET balance = balance + {} WHERE id = {}
            """.format(amount, to_user_id))  # уязвимость: SQL-инъекция
```

# 5 Разработка тестов

## 5.1 Структура проекта и фикстуры

Проект состоит из файлов `auth_manager.py`, `pytest.ini` (регистрация меток), `test_auth_manager.py` (основной набор) и `test_auth_manager_failing.py` (заведомо падающий тест для демонстрации неуспешного прогона). Фикстура `db` создает соединение SQLite в памяти (`:memory:`) и закрывает его после теста, фикстура `auth_manager` создает экземпляр класса на этом соединении; таким образом каждый тест получает чистую базу и тесты не влияют друг на друга. Файл конфигурации приведен в листинге 2.

Листинг 2 – pytest.ini

```
[pytest]
markers =
    basic: базовые тесты (регистрация, аутентификация, удаление)
    parametrized: параметризованные тесты
    exception: тесты на исключения
    security: тесты безопасности (SQL-инъекции)
    slow: медленные тесты
```

### 5.2 Состав тестов

Состав набора приведен в таблице 1. Всего в основном модуле 12 тестовых функций, три из них параметризованы (3, 4 и 4 набора значений), поэтому pytest собирает 20 тестовых узлов; вместе с падающим тестом – 21.

Таблица 1 – Состав тестов

| Тест | Вид | Метка | Что проверяется |
|---|---|---|---|
| test_register_and_get_user_by_id | базовый | basic | после register_user запись находится через get_user_by_id, все поля сохранены |
| test_authenticate_user_success_and_wrong_password | базовый | basic | верная пара логин/пароль → кортеж пользователя; неверный пароль → None |
| test_delete_user | базовый | basic | после delete_user пользователь не находится по id |
| test_count_users_by_country [3 набора] | параметризованный | parametrized | подсчет по стране: CountryA → 2, CountryB → 1, CountryC → 0 |
| test_authenticate_user_parametrized [4 набора] | параметризованный | parametrized | верная пара → найден; неверный пароль, несуществующий логин, другой регистр → None |
| test_transfer_balance_parametrized [4 набора] | параметризованный | parametrized | перевод 200 / 0 / 1000 / 0.5 от user1 (1000) к user2 (500) → ожидаемые балансы |

| Тест | Вид | Метка | Что проверяется |
|---|---|---|---|
| test_transfer_balance_insufficient_funds | исключение | exception | перевод 500 при балансе 100 → ValueError «Insufficient funds», балансы не изменились |
| test_register_duplicate_username | исключение | exception | повторная регистрация того же username → sqlite3.IntegrityError |
| test_sql_injection_authenticate_user | безопасность | security | обход аутентификации payload «admin' OR '1'='1» и «' OR 1=1 --» |
| test_sql_injection_register_user_drop_table | безопасность | security | payload с DROP TABLE → sqlite3.ProgrammingError, таблица сохранена, строк 0 |
| test_register_many_users | пропускаемый | slow + skip | регистрация 10 000 пользователей; пропущен с указанием причины |
| test_user_row_to_dict | условно пропускаемый | skipif (Python < 3.7) | преобразование строки в dict; на Python 3.11 выполняется |
| test_transfer_balance_wrong_expectation (отдельный файл) | заведомо падающий | _ | ожидается баланс 900 вместо реальных 800 |

Полный текст тестового модуля приведен в листинге 3, падающий тест – в листинге 4.

Листинг 3 – test_auth_manager.py

```
"""Практика 3. Тесты класса AuthManager с использованием pytest.

Демонстрируются: фикстуры, метки (маркеры), параметризация,
проверка исключений, пропуск тестов и тесты на SQL-инъекции.
"""
import sqlite3
import sys

import pytest

from auth_manager import AuthManager



# ---------------------------------------------------------------------------
# Фикстуры
# ---------------------------------------------------------------------------
```

```
@pytest.fixture
def db():
    """Соединение с SQLite в памяти; закрывается после каждого теста."""
    connection = sqlite3.connect(":memory:")
    yield connection
    connection.close()



@pytest.fixture
def auth_manager(db):
    """Экземпляр AuthManager, работающий с in-memory базой."""
    return AuthManager(db)



def get_balance(db, user_id):
    """Вспомогательная функция: читает баланс пользователя напрямую из БД."""
    cursor = db.cursor()
    cursor.execute("SELECT balance FROM users WHERE id = ?", (user_id,))
    return cursor.fetchone()[0]



# ---------------------------------------------------------------------------
# Базовые тесты
# ---------------------------------------------------------------------------


@pytest.mark.basic
def test_register_and_get_user_by_id(auth_manager):
    """Регистрация пользователя и получение его по id: поля сохранены верно."""
    auth_manager.register_user("alice", "pass123", "Uzbekistan", 1000)


    user = auth_manager.get_user_by_id(1)


    # Строка: (id, username, password, country, balance)
    assert user is not None
    assert user[0] == 1
    assert user[1] == "alice"
    assert user[2] == "pass123"
    assert user[3] == "Uzbekistan"
    assert user[4] == 1000



@pytest.mark.basic
def test_authenticate_user_success_and_wrong_password(auth_manager):
    """Аутентификация: верный пароль дает пользователя, неверный – None."""
    auth_manager.register_user("bob", "secret", "Russia", 500)


    ok_user = auth_manager.authenticate_user("bob", "secret")
    bad_user = auth_manager.authenticate_user("bob", "wrong")


    assert ok_user is not None
    assert ok_user[1] == "bob"
    assert bad_user is None



@pytest.mark.basic
def test_delete_user(auth_manager):
    """Удаление пользователя: после delete_user он не находится по id."""
    auth_manager.register_user("carol", "qwerty", "Kazakhstan", 300)
    assert auth_manager.get_user_by_id(1) is not None


    auth_manager.delete_user(1)
```

```
    assert auth_manager.get_user_by_id(1) is None


# --------------------------------------------------------------------------
# Параметризованные тесты
# --------------------------------------------------------------------------


@pytest.mark.parametrized
@pytest.mark.parametrize(
    "country, expected_count",
    [
        ("CountryA", 2),
        ("CountryB", 1),
        ("CountryC", 0),
    ],
)
def test_count_users_by_country(auth_manager, country, expected_count):
    """Подсчет пользователей по стране для разных стран (как в методичке)."""
    auth_manager.register_user("user1", "password123", "CountryA", 1000)
    auth_manager.register_user("user2", "password123", "CountryA", 1000)
    auth_manager.register_user("user3", "password123", "CountryB", 1000)

    assert auth_manager.count_users_by_country(country) == expected_count



@pytest.mark.parametrized
@pytest.mark.parametrize(
    "username, password, expect_found",
    [
        ("dave", "pass", True),        # верная пара
        ("dave", "wrong", False),      # неверный пароль
        ("nobody", "pass", False),     # несуществующий логин
        ("DAVE", "pass", False),       # регистр логина важен
    ],
)
def test_authenticate_user_parametrized(auth_manager, username, password,
                                        expect_found):
    """Аутентификация с разными парами логин/пароль → найден или None."""
    auth_manager.register_user("dave", "pass", "Country", 100)

    user = auth_manager.authenticate_user(username, password)

    assert (user is not None) is expect_found



@pytest.mark.parametrized
@pytest.mark.parametrize(
    "amount, expected_from, expected_to",
    [
        (200, 800, 700),    # пример из методички
        (0, 1000, 500),     # нулевой перевод ничего не меняет
        (1000, 0, 1500),    # перевод всего баланса
        (0.5, 999.5, 500.5),  # дробная сумма
    ],
)
def test_transfer_balance_parametrized(auth_manager, db, amount,
                                       expected_from, expected_to):
    """Перевод разных сумм: балансы отправителя и получателя корректны."""
    auth_manager.register_user("user1", "password123", "CountryA", 1000)
    auth_manager.register_user("user2", "password123", "CountryB", 500)

    auth_manager.transfer_balance(1, 2, amount)

    assert get_balance(db, 1) == expected_from
    assert get_balance(db, 2) == expected_to
```

```
# ------------------------------------------------------------------------
# Тесты исключений
# ------------------------------------------------------------------------


@pytest.mark.exception
def test_transfer_balance_insufficient_funds(auth_manager, db):
    """Перевод суммы больше баланса → ValueError("Insufficient funds"),
    балансы при этом не меняются."""
    auth_manager.register_user("poor", "p", "Country", 100)
    auth_manager.register_user("rich", "p", "Country", 1000)


    with pytest.raises(ValueError, match="Insufficient funds"):
        auth_manager.transfer_balance(1, 2, 500)


    assert get_balance(db, 1) == 100
    assert get_balance(db, 2) == 1000



@pytest.mark.exception
def test_register_duplicate_username(auth_manager):
    """Повторная регистрация того же username → sqlite3.IntegrityError
    (нарушение ограничения UNIQUE)."""
    auth_manager.register_user("eve", "p1", "Country", 10)


    with pytest.raises(sqlite3.IntegrityError,
                       match="UNIQUE constraint failed"):
        auth_manager.register_user("eve", "p2", "Country", 20)



# ------------------------------------------------------------------------
# Тесты безопасности: SQL-инъекции
# ------------------------------------------------------------------------


@pytest.mark.security
def test_sql_injection_authenticate_user(auth_manager):
    """Доказательство уязвимости: authenticate_user с payload
    "admin' OR '1'='1" возвращает пользователя БЕЗ знания пароля."""
    auth_manager.register_user("admin", "supersecret", "Country", 1000)


    # Итоговый SQL: ... WHERE username = 'admin' OR '1'='1' AND password = 'x'
    # AND имеет более высокий приоритет, чем OR, поэтому условие читается как
    # username = 'admin' OR ('1'='1' AND password = 'x') – первая часть
    # истинна, и строка admin возвращается без проверки пароля.
    user = auth_manager.authenticate_user("admin' OR '1'='1", "x")


    assert user is not None
    assert user[1] == "admin"


    # Усиленный payload: "--" комментирует проверку пароля целиком,
    # пользователь возвращается даже без знания логина.
    any_user = auth_manager.authenticate_user("' OR 1=1 --", "x")
    assert any_user is not None
    assert any_user[1] == "admin"


    # Контроль: с обычным неверным паролем доступа нет
    assert auth_manager.authenticate_user("admin", "x") is None



@pytest.mark.security
def test_sql_injection_register_user_drop_table(auth_manager, db):
    """Попытка инъекции "DROP TABLE users" через register_user.


    Фактическое поведение sqlite3: метод execute() выполняет только ОДНО
```

```
    выражение, поэтому строка с двумя statement отвергается с
    sqlite3.ProgrammingError ("You can only execute one statement at a
    time"). Таблица users при этом остается на месте.
    """
    payload = "x', 'p', 'c', 0); DROP TABLE users; --"

    with pytest.raises(sqlite3.ProgrammingError,
                       match="only execute one statement"):
        auth_manager.register_user(payload, "p", "c", 0)

    cursor = db.cursor()
    cursor.execute(
        "SELECT name FROM sqlite_master WHERE type='table' AND name='users'"
    )
    tables = cursor.fetchall()
    assert len(tables) == 1  # таблица users все еще существует

    # Ни одна строка не вставлена – INSERT не был выполнен
    cursor.execute("SELECT COUNT(*) FROM users")
    assert cursor.fetchone()[0] == 0


# ---------------------------------------------------------------------------
# Пропускаемые тесты
# ---------------------------------------------------------------------------

@pytest.mark.slow
@pytest.mark.skip(reason="Медленный тест: массовая регистрация, "
                         "не запускается в учебном прогоне")
def test_register_many_users(auth_manager):
    """Регистрация 10 000 пользователей (демонстрация @pytest.mark.skip)."""
    for i in range(10_000):
        auth_manager.register_user(f"user{i}", "p", "Country", i)
    assert auth_manager.count_users_by_country("Country") == 10_000


@pytest.mark.skipif(sys.version_info < (3, 7),
                    reason="Требуется Python 3.7+ (упорядоченные dict)")
def test_user_row_to_dict(auth_manager):
    """Преобразование строки пользователя в dict (демонстрация skipif:
    на Python 3.7+ тест выполняется, на более старых – пропускается)."""
    auth_manager.register_user("frank", "p", "Country", 42)
    row = auth_manager.get_user_by_id(1)
    keys = ("id", "username", "password", "country", "balance")

    user = dict(zip(keys, row))

    assert list(user) == list(keys)
    assert user["username"] == "frank"
    assert user["balance"] == 42
```

## Листинг 4 – test_auth_manager_failing.py

```
"""Заведомо падающий тест: демонстрация красного прогона pytest.

Ожидаемое значение указано НЕВЕРНО специально, чтобы увидеть,
как pytest показывает ошибку assert с диффом.
"""
import sqlite3

import pytest

from auth_manager import AuthManager
```

```
@pytest.fixture
def db():
    """Соединение с SQLite в памяти; закрывается после теста."""
    connection = sqlite3.connect(":memory:")
    yield connection
    connection.close()



@pytest.fixture
def auth_manager(db):
    """Экземпляр AuthManager на in-memory базе."""
    return AuthManager(db)



def test_transfer_balance_wrong_expectation(auth_manager, db):
    """После перевода 200 от user1 (1000) к user2 (500) ожидаем 900 у user1 -
    это НЕВЕРНО (правильно 800), тест должен упасть."""
    auth_manager.register_user("user1", "password123", "CountryA", 1000)
    auth_manager.register_user("user2", "password123", "CountryB", 500)


    auth_manager.transfer_balance(1, 2, 200)


    cursor = db.cursor()
    cursor.execute("SELECT balance FROM users WHERE id = 1")
    balance1 = cursor.fetchone()[0]


    assert balance1 == 900  # намеренная ошибка: реальный баланс 800
```

### 5.3 Тесты на SQL-инъекции

Вызов `authenticate_user("admin' OR '1'='1", "x")` формирует запрос `SELECT * FROM users WHERE username = 'admin' OR '1'='1' AND password = 'x'`. Поскольку `AND` имеет более высокий приоритет, чем `OR`, условие читается как `username = 'admin' OR ('1'='1' AND password = 'x')` – первая часть истинна, и запись администратора возвращается без проверки пароля. Усиленный payload `' OR 1=1 --` комментирует проверку пароля целиком и возвращает первого пользователя даже без знания логина. Тест закрепляет оба факта и контрольно проверяет, что обычный неверный пароль по-прежнему отклоняется.

Вторая инъекция – попытка удалить таблицу через `register_user` с payload `x', 'p', 'c', 0); DROP TABLE users; --`. Фактическое поведение отличается от наивного ожидания: метод `sqlite3.Connection.execute()` выполняет ровно одно SQL-выражение и отвергает строку с двумя выражениями

исключением `sqlite3.ProgrammingError: You can only execute one statement at a time`. Таблица остается в `sqlite_master`, а вставка не выполняется, так как транзакция `with self.connection` откатывается. Payload из примера методических указаний `testuser'; DROP TABLE users; --` дает `sqlite3.OperationalError: near ";": syntax error` из-за незакрытой скобки `VALUES (`. Следовательно, защита от многовыраженной инъекции здесь является свойством драйвера DB-API, а не кода класса: инъекции внутри одного выражения (обход аутентификации) работают, и класс требует перехода на параметризованные запросы.

## 6 Результаты выполнения тестов

Тесты выполнялись в окружении Python 3.11.15, pytest 9.1.1, SQLite 3.45.1. Команды запуска и итоги приведены в таблице 2.

Таблица 2 – Прогоны тестов

| № | Команда | Результат | Рисунок |
|---|---|---|---|
| 1 | `python -m pytest test_auth_manager.py -v --maxfail=2 --tb=short` | 19 passed, 1 skipped | 1 |
| 2 | `python -m pytest test_auth_manager.py test_auth_manager_failing.py -v --maxfail=2 --tb=short` | 1 failed, 19 passed, 1 skipped | 2 |
| 3 | `python -m pytest -m "basic or exception" -v` | 5 passed, 16 deselected | 3 |
| 4 | `python -m pytest --collect-only -q` | 21 tests collected | 4 |

Правильный результат – успешный прогон основного модуля (рисунок 1): все 19 тестов пройдены, один пропущен с указанием причины, тесты на SQL-инъекции подтвердили уязвимость метода `authenticate_user`.

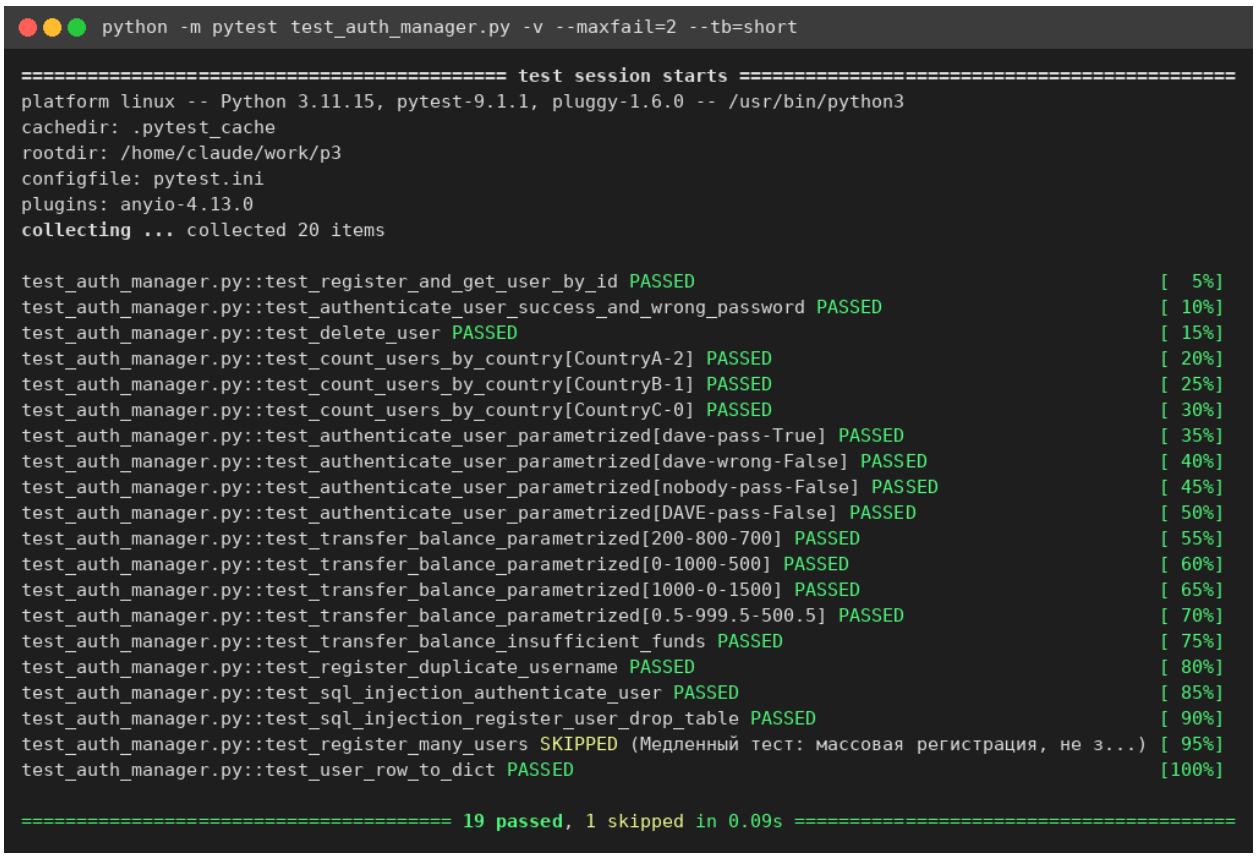



Рисунок 1 – Успешный прогон: 19 passed, 1 skipped

Неправильный результат – прогон с заведомо неверным ожиданием (рисунок 2): pytest показывает упавший тест, строку с `assert balance1 == 900`, фактическое значение `800.0` и итоговую строку `1 failed, 19 passed, 1 skipped`. Опция `--maxfail=2` позволила бы продолжить выполнение до двух неудач; поскольку неудача одна, сессия завершилась штатно после всех тестов.

```
python -m pytest test_auth_manager.py test_auth_manager_failing.py -v --maxfail=2 --tb=short

============================================ test session starts ============================================
platform linux -- Python 3.11.15, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/claude/work/p3
configfile: pytest.ini
plugins: anyio-4.13.0
collecting ... collected 21 items

test_auth_manager.py::test_register_and_get_user_by_id PASSED                                      [  4%]
test_auth_manager.py::test_authenticate_user_success_and_wrong_password PASSED                     [  9%]
test_auth_manager.py::test_delete_user PASSED                                                      [ 14%]
test_auth_manager.py::test_count_users_by_country[CountryA-2] PASSED                               [ 19%]
test_auth_manager.py::test_count_users_by_country[CountryB-1] PASSED                               [ 23%]
test_auth_manager.py::test_count_users_by_country[CountryC-0] PASSED                               [ 28%]
test_auth_manager.py::test_authenticate_user_parametrized[dave-pass-True] PASSED                   [ 33%]
test_auth_manager.py::test_authenticate_user_parametrized[dave-wrong-False] PASSED                 [ 38%]
test_auth_manager.py::test_authenticate_user_parametrized[nobody-pass-False] PASSED                [ 42%]
test_auth_manager.py::test_authenticate_user_parametrized[DAVE-pass-False] PASSED                  [ 47%]
test_auth_manager.py::test_transfer_balance_parametrized[200-800-700] PASSED                       [ 52%]
test_auth_manager.py::test_transfer_balance_parametrized[0-1000-500] PASSED                        [ 57%]
test_auth_manager.py::test_transfer_balance_parametrized[1000-0-1500] PASSED                       [ 61%]
test_auth_manager.py::test_transfer_balance_parametrized[0.5-999.5-500.5] PASSED                   [ 66%]
test_auth_manager.py::test_transfer_balance_insufficient_funds PASSED                              [ 71%]
test_auth_manager.py::test_register_duplicate_username PASSED                                      [ 76%]
test_auth_manager.py::test_sql_injection_authenticate_user PASSED                                  [ 80%]
test_auth_manager.py::test_sql_injection_register_user_drop_table PASSED                           [ 85%]
test_auth_manager.py::test_register_many_users SKIPPED (Медленный тест: массовая регистрация, не з...) [ 90%]
test_auth_manager.py::test_user_row_to_dict PASSED                                                 [ 95%]
test_auth_manager_failing.py::test_transfer_balance_wrong_expectation FAILED                       [100%]

================================================= FAILURES ==================================================
_____________________________ test_transfer_balance_wrong_expectation _____________________________
test_auth_manager_failing.py:39: in test_transfer_balance_wrong_expectation
    assert balance1 == 900  # намеренная ошибка: реальный баланс 800
    ^^^^^^^^^^^^^^^^^^^^^^
E   assert 800.0 == 900
========================================== short test summary info ==========================================
FAILED test_auth_manager_failing.py::test_transfer_balance_wrong_expectation - assert 800.0 == 900
================================== 1 failed, 19 passed, 1 skipped in 0.08s ==================================
```

Рисунок 2 – Неуспешный прогон: 1 failed, видна диагностика assert

Отбор по меткам (рисунок 3) выбрал пять тестов с метками `basic` и `exception`, остальные 16 узлов отклонены (deselected); список всех собранных узлов с идентификаторами параметризации показан на рисунке 4.

```
python -m pytest -m "basic or exception" -v

============================================ test session starts ============================================
platform linux -- Python 3.11.15, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/claude/work/p3
configfile: pytest.ini
plugins: anyio-4.13.0
collecting ... collected 21 items / 16 deselected / 5 selected

test_auth_manager.py::test_register_and_get_user_by_id PASSED                                      [ 20%]
test_auth_manager.py::test_authenticate_user_success_and_wrong_password PASSED                     [ 40%]
test_auth_manager.py::test_delete_user PASSED                                                      [ 60%]
test_auth_manager.py::test_transfer_balance_insufficient_funds PASSED                              [ 80%]
test_auth_manager.py::test_register_duplicate_username PASSED                                      [100%]

=================================== 5 passed, 16 deselected in 0.02s ====================================
```

Рисунок 3 – Отбор тестов по меткам: -m "basic or exception"

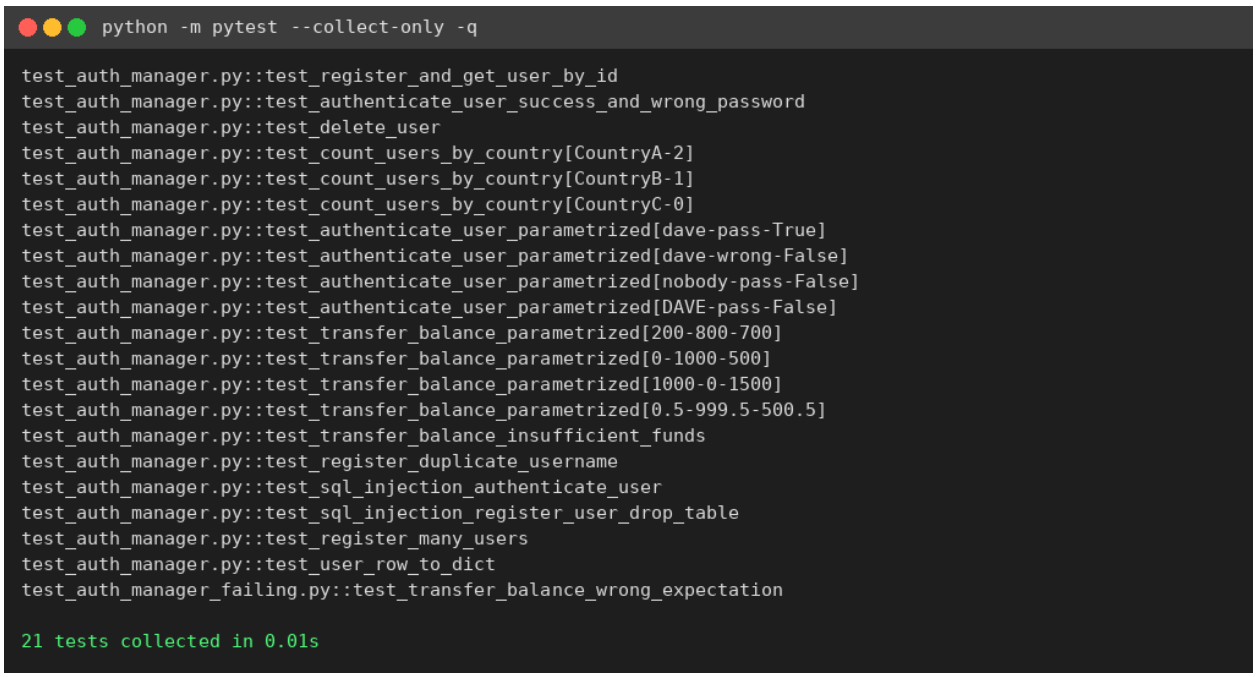



Рисунок 4 – Сбор тестов: 21 узел, включая параметризованные

## 7 Выводы

В работе разработан набор из 21 тестового узла для класса `AuthManager`, продемонстрировавший все требуемые возможности pytest: фикстуры с in-memory базой SQLite, три базовых теста, три параметризованных теста (11 комбинаций), два теста исключений с проверкой текста сообщений, пять зарегистрированных меток и отбор по ним, пропуск тестов декораторами `skip` и `skipif`, запуск с `--maxfail`. Получены снимки успешного и неуспешного прогонов.

Тесты безопасности доказали уязвимость метода `authenticate_user` к обходу аутентификации через SQL-инъекцию, а также показали, что попытка удалить таблицу через второе SQL-выражение блокируется драйвером sqlite3, а не кодом класса. Для устранения уязвимости все методы класса необходимо перевести на параметризованные запросы с плейсхолдерами.